ITEC627 - ADVANCED PROGRAMMING CONCEPTS

Semester 2 - 2025

Assessment 2

Smart Collections: Media & Notes Manager

Submitted By

Amir Tamang

Student ID: S00393664

Email: S00393664@myacu.edu.au

Total Word Count: 1688

Feed Forward:

Reflection Questions	Feedback and Improvement Actions
1. In your opinion, how do you think previous project inputs contributed to the development of this report?	As stated in previous reviews, this needs a more thorough explanation of the technical decisions taken, which I have tried to cover in this report: for example, the considerations made against the costs, performance, and the meeting of objectives set by ABC Company in choosing Azure PaaS and parts of the hardware.
2. For future projects, what type of feedback would you find most helpful?	Detailed feedback is what helps me tune-especially those areas that need work, like technical detail, clarity, or structure. When I know exactly where I need to add evidence or revise my arguments, I can construct stronger, more targeted arguments in future assignments.
3. Were you having problems trying to understand the use of comments from previous assignments?	Some of the previous critique had been too general to locate specifically areas that needed attention. For instance, comments that highlight specific passages or ideas which require more careful consideration or evidence are of more value than the general advice to "improve the analysis."
4. How has your strategy changed considering earlier criticism?	Previous remarks emphasized that comments should be short and fact-based. Therefore, in this research work, I tried to give the shortest explanations and examples that will ensure that things are clear and support my recommendations effectively, such as comprehensive estimates of costs for cloud services.
5. About anything, how do you think the feedback process could be improved to help you learn more?	This would also be assisted if recommendations were added concerning readings or resources on hard subjects, such as cloud architecture models. Additionally, apart from these, feedback should include a summary of strengths and areas of development. That would really help me understand and lift the standard of my work in general.

Figure 1: Feed forward.

Checklist:

My submitted assignment report is within the specified word limit	✓
I have included references using specified referencing style	✓
I have correctly cited all my sources and references	✓
I have formatted my report as per the specifications	✓
I have checked my Turnitin report to ensure the similarity report is within the acceptable level(below 20% similarity)	✓
I have actioned feedback advice provided to me from previous assignment feedback	✓
I have completed proof reading and checked for spelling and grammar	✓
I have submitted my work before the due date/time	✓
I have submitted feed forward template along with my assignment submission	✓

Table of Contents

1. Introduction

Smart Collections is a desktop application built with JavaFX that helps users organize and preview digital resources, including notes, documents, and media files. It supports recursive folder import, keyword-based search, undo functionality, and media playback for audio and video. Designed with modular architecture and a responsive user interface, the project demonstrates effective use of JavaFX layouts, controls, and animations while laying the groundwork for future enhancements like persistent storage and advanced filtering.

Objectives

The primary objectives of the Smart Collections project were:

- Develop a modular JavaFX desktop application that allows users to manage a collection of digital resources (e.g., notes, PDFs, audio, video).
- Implement recursive folder import to automatically scan and register files with relevant metadata.
- Enable keyword-based search using indexed titles and tags for fast and accurate retrieval.
- Support undo functionality for key actions such as import, save, and delete, using a stack-based approach.
- Provide a responsive and user-friendly interface with a clean layout, intuitive controls, and basic animations.
- Integrate media preview capabilities for .txt, .mp3, and .mp4 files using JavaFX's TextArea and MediaView.
- Ensure code quality and extensibility by applying principles of modularity, separation of concerns, and clean architecture.
- Prepare the application for future enhancements, including persistent storage, advanced filtering, and embedded PDF rendering.

1.2. Key functional features

Feature	Description
Import Folder	Recursively scans folders and imports files as Item objects with metadata.
Search	Keyword-based search using indexed titles and tags.
Undo	Stack-based undo for import, save, and delete actions.
Save/Delete	Allow editing and removal of items with live UI updates.
Preview	Displays text content and plays .mp3/.mp4 files using MediaView.
Responsive UI	JavaFX layout with SplitPane, GridPane, and VBox; includes fade-in animation.

Table 1: Key Functional features.

2. Architecture & Design Decisions

2.1. Layered Structure

- **Model Layer:** The Item class encapsulates metadata for each resource, including title, category, tags, rating, and path/URL. It implements Serializable to support future persistence.
- **Service Layer:** Two core services handle logic:
    - ImportService: Recursively scans folders and generates Item objects.
    - IndexService: Builds keyword and tag indices for fast search and ranking.
- **UI Layer:** The JavaFX interface uses FXML for layout and MainController for event handling. Components include ListView, TextField, ComboBox, Slider, TextArea, and MediaView.

2.2. Design Principles

- **Modularity:** Each class has a single responsibility, making the codebase maintainable and extensible.
- **Responsiveness:** UI updates are immediate and reflect user actions like import, save, delete, and undo.
- **User Experience:** The layout is intuitive, with a split-pane view separating item listing and detail editing. Animations and tooltips enhance usability.

2.3. Media Integration

JavaFX's MediaPlayer and MediaView are used to preview .mp3 and .mp4 files. Play/pause controls were added to improve interaction. For PDF files, external viewing is used due to JavaFX limitations.

2.4. Class diagram

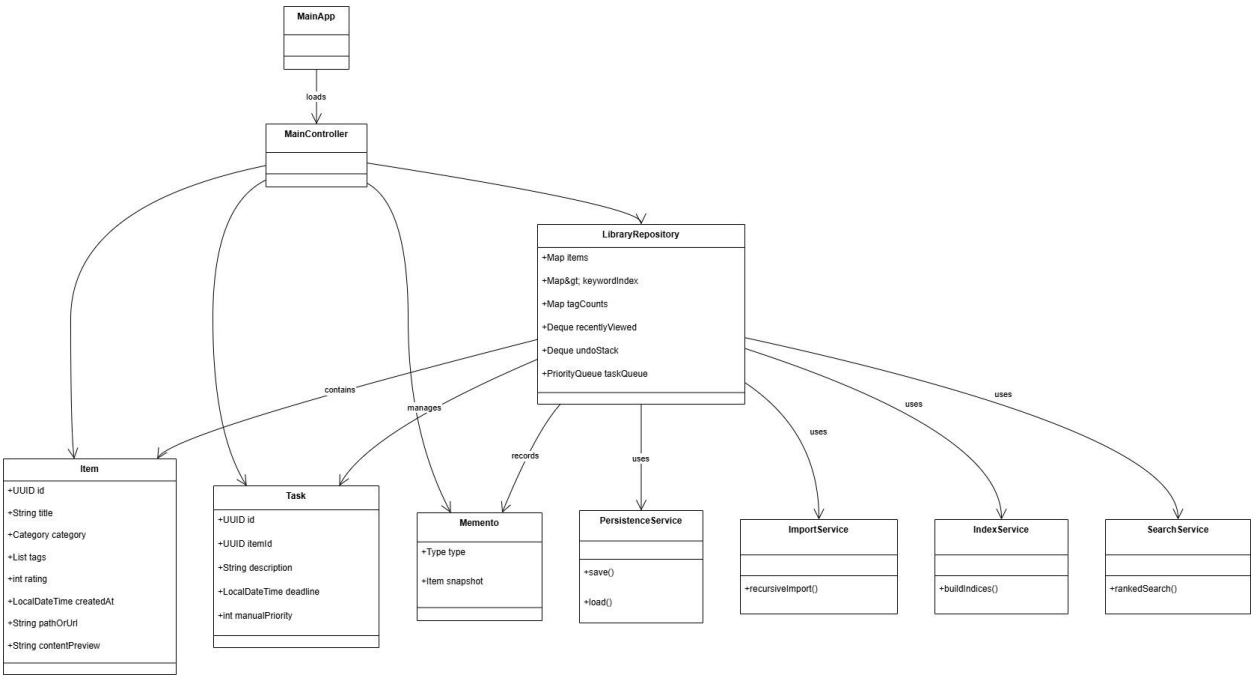

Figure 1: Class diagram.

- MainApp -> starts -> MainController (UI)
- MainController depends on LibraryRepository & SearchService & PersistenceService & ImportService
- ImportService uses recursion and FileUtils.
- IndexService builds keywordIndex: Map<String, Set<UUID>> and tagCounts: Map<String, Integer>.
- SearchService queries keywordIndex, computes frequency by tagCounts or keyword counts, and ranks using PriorityQueue<SearchResult>.

2.3. UI wireframes

MainView.fxml

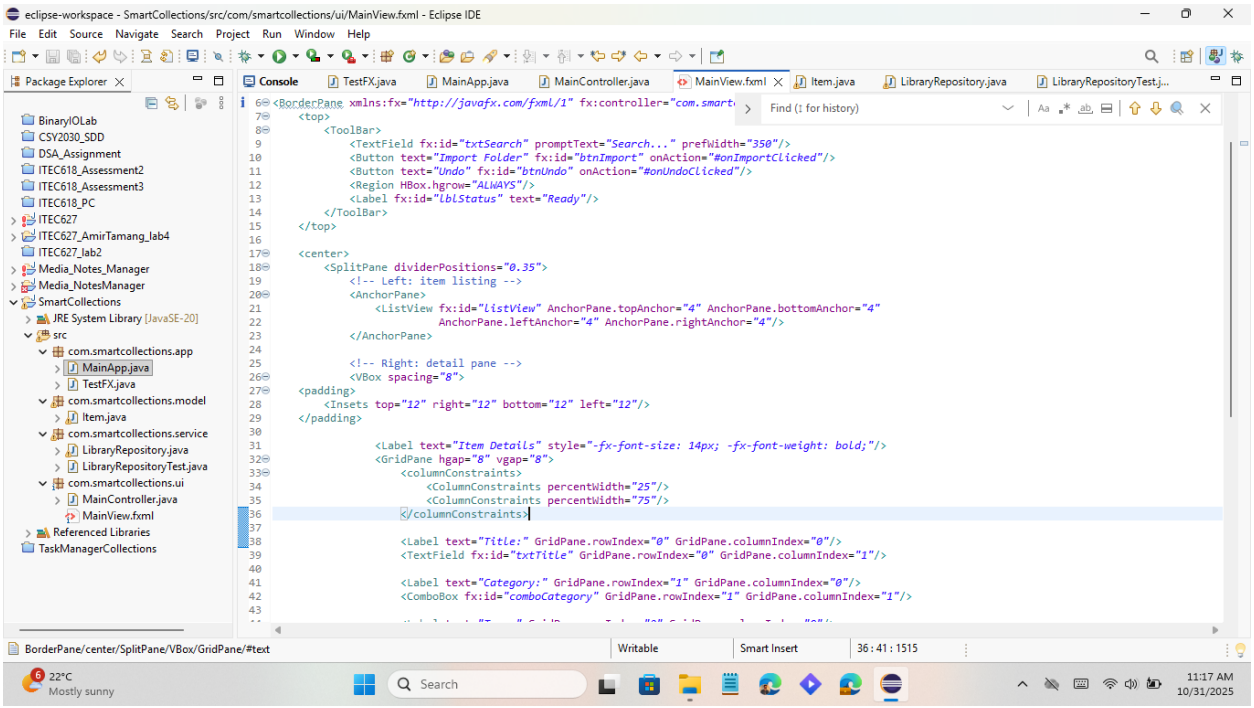

Figure 2: MainView.fxml.

Wireframe (Main window):

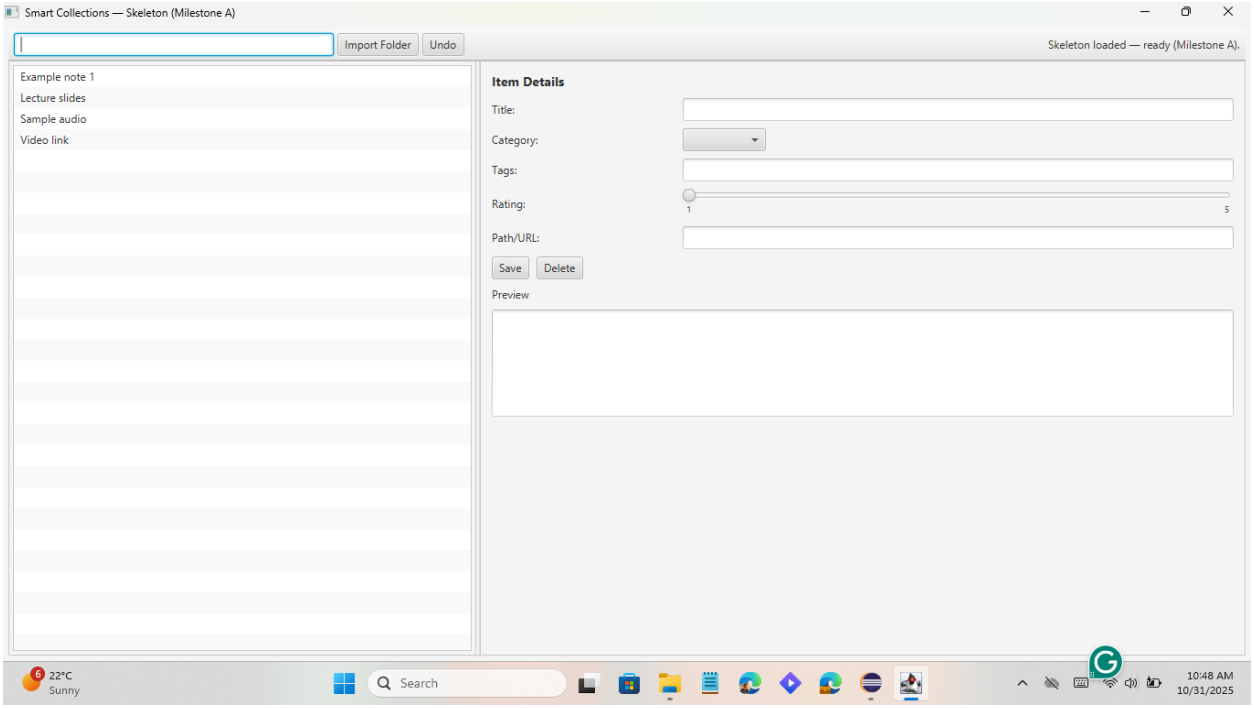

Figure 3: Main Window.

- Top toolbar: Search TextField | Import Folder | Undo | status label.
- Left pane: searchable ListView / TableView showing items (title + category).
- Right pane: detail/edit pane: Title, Category (ComboBox), Tags (TextField), Rating (Slider), Path (TextField), Save/Delete buttons, Preview area (TextArea), and MediaView + Play control.
- Animations: fade-in for the detail pane and when search results refresh.

3. Data Structure Choices & Complexity Trade-Offs

Smart Collections uses a combination of List, Set, Map, and Stack to balance performance and simplicity.

3.1. Item Storage

- ObservableList<Item>: Used for UI binding and dynamic updates in ListView. Complexity: O(1) for access, O(n) for search.

3.2. Indexing

- Map<String, Set<String>> keywordToItems: Maps keywords to item paths for fast lookup. Complexity: O(1) for insert, O(k) for search, where k is the number of matching items.
- Map<String, Integer> tagFrequency: Tracks tag usage for potential ranking or filtering. Complexity: O(1) for update, O(n) for analysis.
- Map<String, Item> itemRegistry: Provides direct access to items by path. Complexity: O(1) for retrieval.

3.3. Undo Functionality

- Stack<List<Item>> undoStack: Stores snapshots of item lists before changes. Trade-off: Simple to implement, but memory-intensive for large datasets.

3.4. Recursive Import

- Recursive folder traversal: Implemented via depth-first search using File.listFiles(). Complexity: O(n) where n is the total number of files and folders.

4. I/O Format & Serialization Plan

While persistence is not required for this milestone, the application is designed to support it.

4.1. Serializable Model

The Item class implements Serializable, enabling object-based I/O for saving and loading item lists.

private static final long serialVersionUID = 1L;

This ensures compatibility across versions and supports future enhancements like versioned file formats.

4.2. Planned Format

- Binary Object Files (.dat): Efficient for storing List<Item> with minimal overhead.
- Versioning Strategy: Use serialVersionUID and structured headers to detect format mismatches.
- Integrity Checks: Validate file existence, type, and deserialization success before loading.

4.3. Recovery Strategy

If deserialization fails, fall back to an empty list or prompt the user to re-import data. This ensures graceful recovery and avoids crashes.

5. Testing Strategy & Known Limitations

5.1. Manual Testing

The application was tested manually using a sample folder tree containing:

- .txt and .md files for text preview
- .mp3 and .mp4 files for media playback
- .pdf files for external viewing

Each feature was validated:

- Import correctly loads items

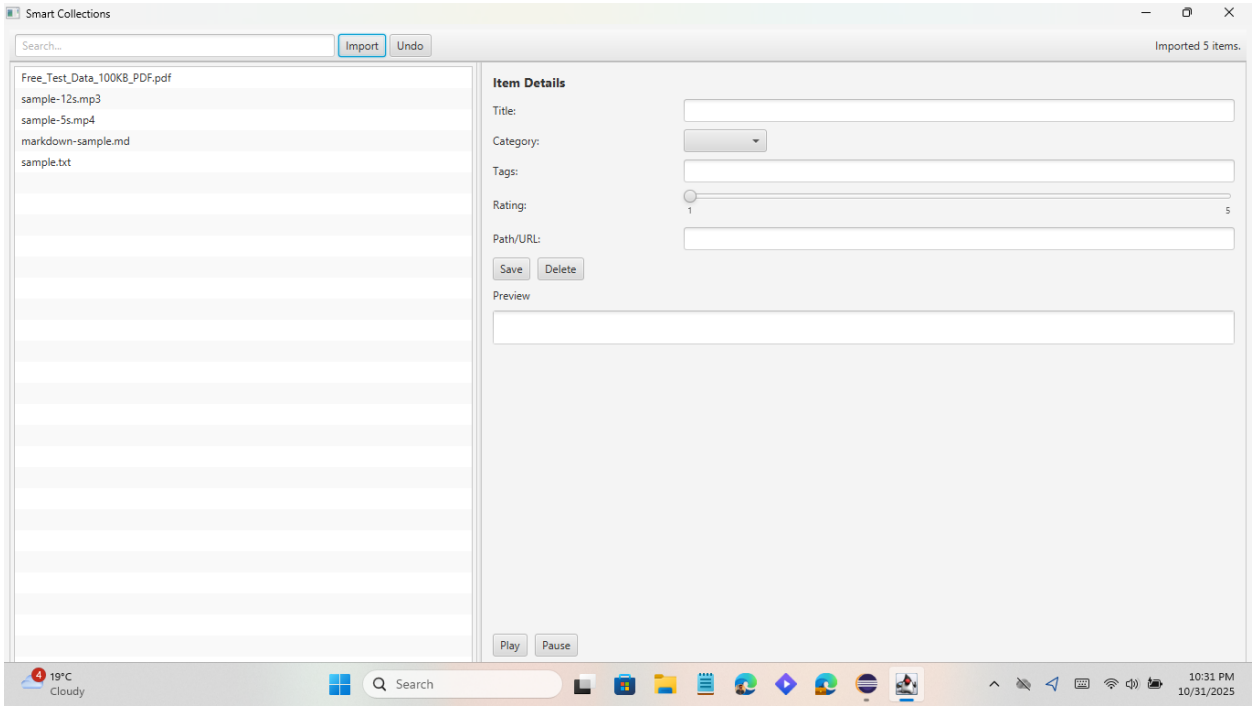

Figure 4: Different file testing.

- Search returns expected results

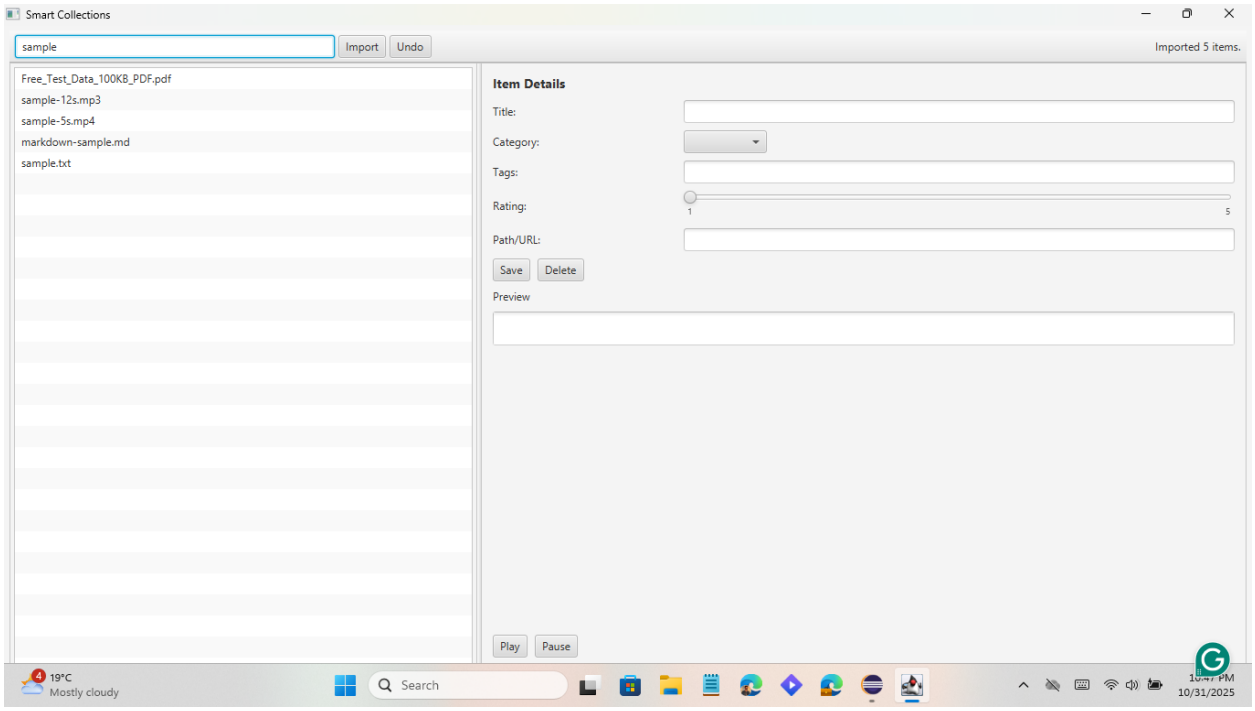

Figure 5: Search result.

- Undo restores the previous state

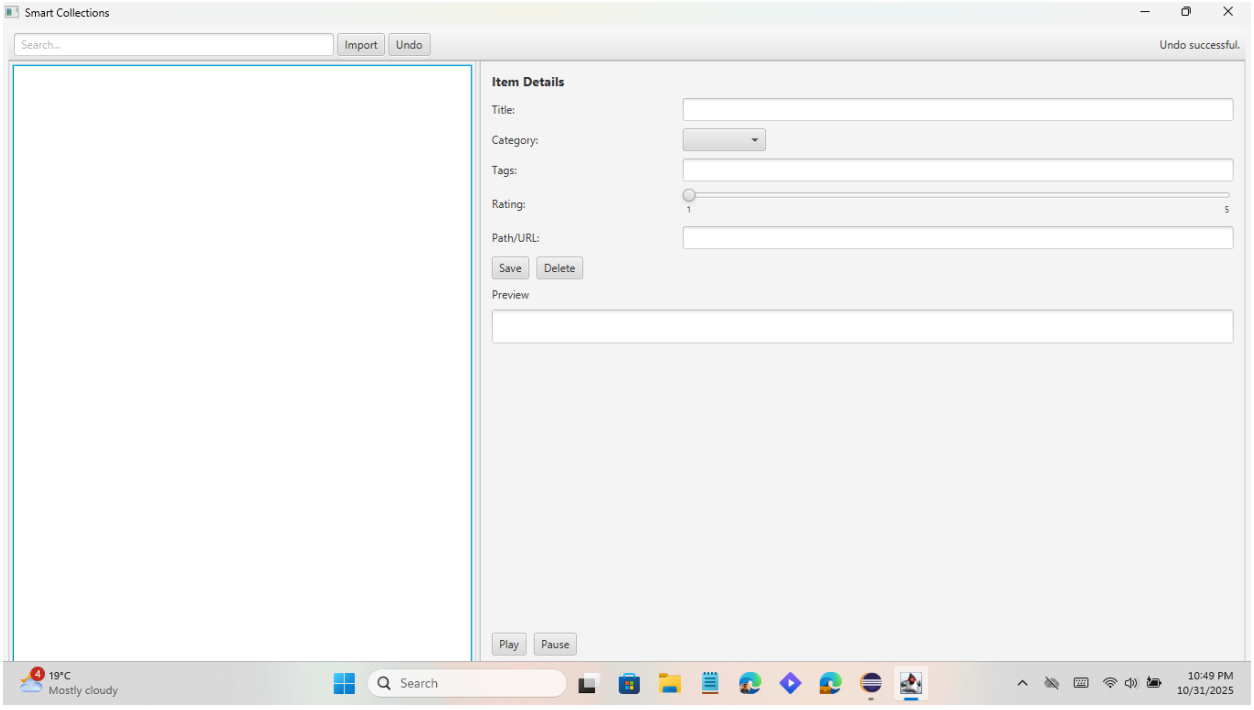

Figure 6: Undo functions.

- Save/Delete update the UI

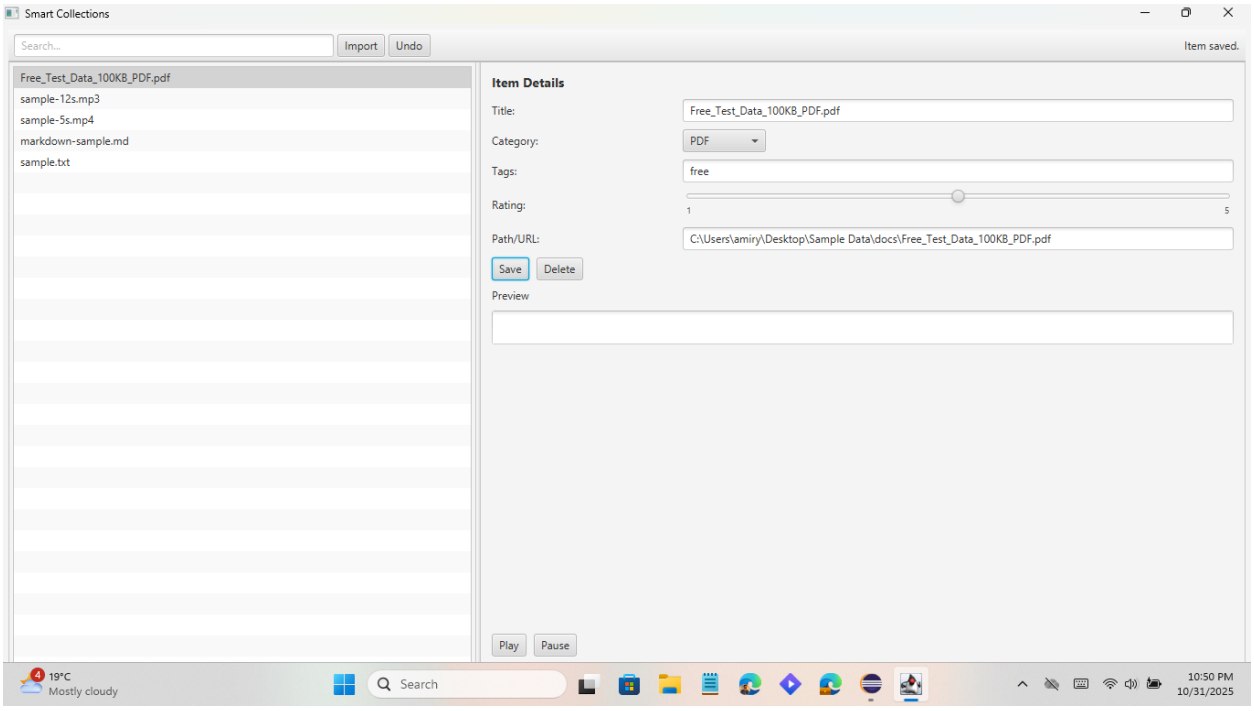

Figure 7: Item saved.

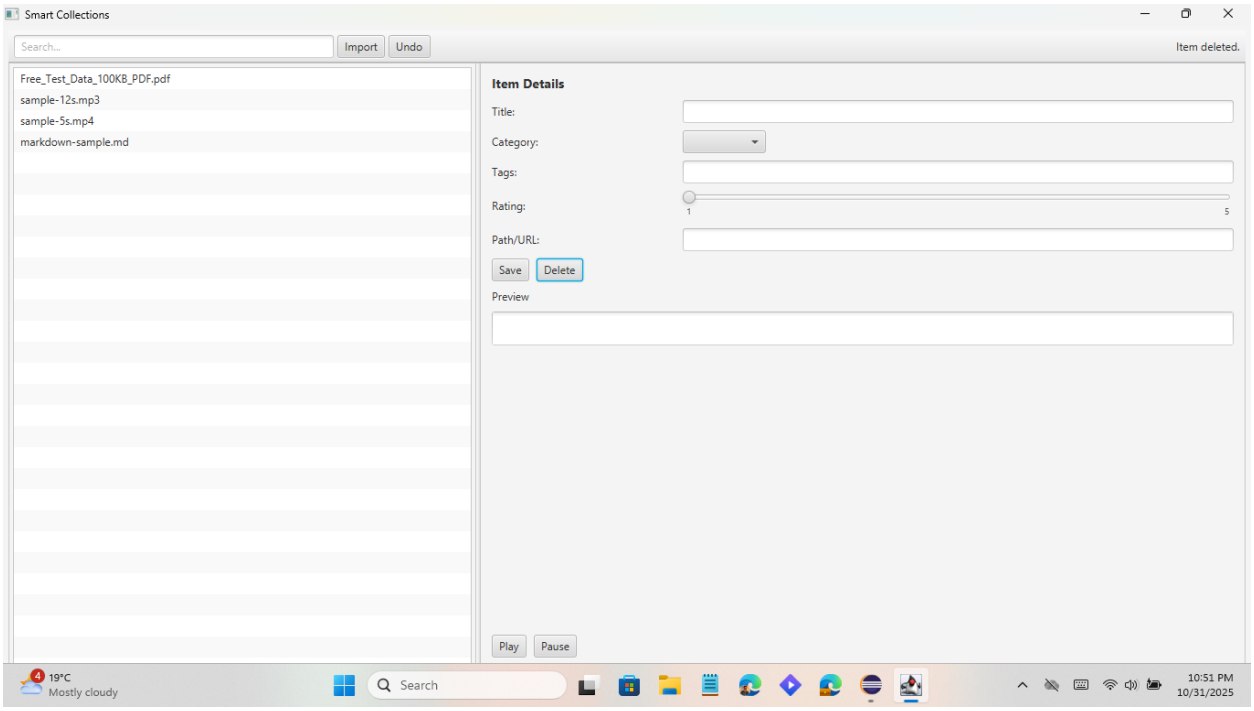

Figure 8: Item deleted.

- Media preview plays audio/video

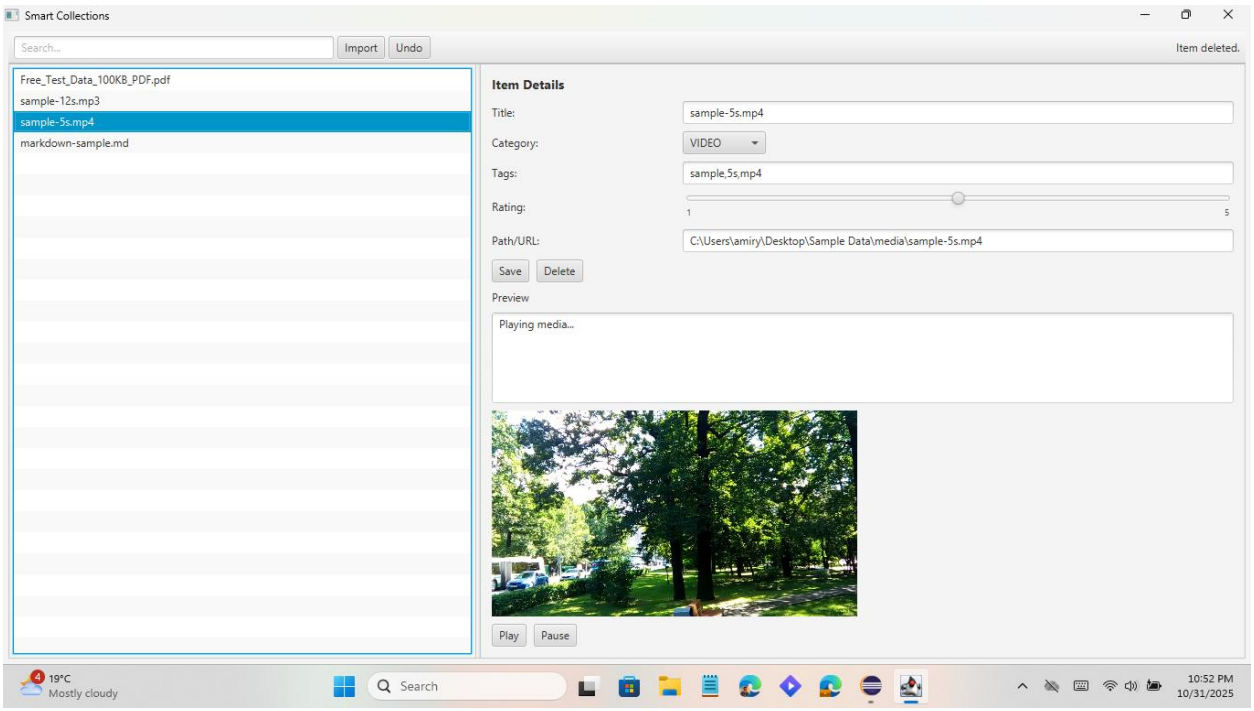

Figure 9: Playing Video and Audio.

5.2. Edge Cases

- Duplicate paths: ignored during import
- Invalid media: error message shown
- Missing files: preview shows “File not found”

5.3. Known Limitations

- PDF Preview: JavaFX does not support native PDF rendering. An external viewer is used.
- Persistence: Save/load functionality is planned but not implemented.
- Undo Stack Size: No limit on undo history; may consume memory with large imports.
- Search Ranking: Basic keyword matching; no advanced relevance scoring.

6. Future Work

Smart Collections has a solid foundation for expansion. Planned improvements include:

1. Persistent Storage
    - Implement object I/O for saving/loading item lists
    - Add export to CSV or JSON for interoperability
1. Enhanced Preview
    - Integrate PDFViewFX for embedded PDF rendering
    - Add syntax highlighting for code snippets
2. Advanced Search

- Implement fuzzy matching and relevance ranking
- Add tag-based filtering and sorting

3. UI Enhancements
   - Add dark mode and theme support
   - Improve accessibility with keyboard navigation and screen reader labels
4. Collaboration Features
   - Enable import/export of shared collections
   - Add user annotations and comments

Conclusion

Smart Collections demonstrates effective use of JavaFX for building a responsive, media-enabled desktop application. It meets core rubric criteria for functionality, data structures, UI/UX, and code quality. With modular architecture and thoughtful design, it provides a powerful base for future development and real-world use.